

JavaScript — Complete 50-Page Notes

Fundamentals → Advanced JavaScript → Node.js → Playwright/SDET

Practical explanations, syntax, examples, interview points and coding practice.

Page 1: JavaScript Overview

What JavaScript is, ECMAScript, engines, browser vs Node.js, and why it matters for automation.

Key points

- **JavaScript is dynamically typed, supports multiple programming paradigms, and is standardized as ECMAScript. Modern test automation frequently uses JavaScript with Node.js.**

Practical example

```
console.log('Hello JavaScript');  
console.log(2 + 3);
```

Interview checkpoint: explain this topic in your own words and identify one real automation use case.

Page 2: Node.js, npm & VS Code

Installing Node.js, checking versions, npm init, package.json, scripts, dependencies and project structure.

Key points

- **Use Node.js LTS, npm, VS Code and a clean project structure. package.json stores scripts and dependency declarations.**

Practical example

```
node -v
npm -v
npm init -y
npm install @playwright/test
```

Interview checkpoint: explain this topic in your own words and identify one real automation use case.

Page 3: Variables

var, let, const, reassignment, declaration and best practices.

Key points

- **Prefer const by default. Use let for reassignment. Avoid var in modern application and test code.**

Practical example

```
const name = 'Asha';  
let count = 1;  
count = 2;
```

Interview checkpoint: explain this topic in your own words and identify one real automation use case.

Page 4: Data Types

string, number, bigint, boolean, undefined, null, symbol and object.

Key points

- **Know primitive types and objects. Remember the historical `typeof null === 'object'` behavior.**

Practical example

```
console.log(typeof 'QA');  
console.log(typeof 42);  
console.log(typeof true);  
console.log(typeof null);
```

Interview checkpoint: explain this topic in your own words and identify one real automation use case.

Page 5: Primitive vs Reference

Value copying, object references, mutation, shallow copies and common interview traps.

Key points

- **Objects and arrays are reference values. Spread creates shallow copies, not automatic deep copies.**

Practical example

```
const a = {score: 10};  
const b = a;  
b.score = 20;  
console.log(a.score); // 20
```

Interview checkpoint: explain this topic in your own words and identify one real automation use case.

Page 6: Operators

Arithmetic, assignment, comparison, logical, nullish, optional chaining and exponentiation.

Key points

- Use `===` for predictable equality. Learn `&&`, `||`, `??` and `?`. because they appear frequently in production code.

Practical example

```
console.log(10 % 3);  
console.log(2 ** 3);  
console.log(true && false);
```

Interview checkpoint: explain this topic in your own words and identify one real automation use case.

Page 7: Equality & Type Coercion

==, ===, implicit conversion and predictable comparison practices.

Key points

- **== coerces types; === does not. Explain examples instead of memorizing isolated rules.**

Practical example

```
console.log(5 == '5'); // true
console.log(5 === '5'); // false
```

Interview checkpoint: explain this topic in your own words and identify one real automation use case.

Page 8: Truthy, Falsy & Nullish

Falsy values, Boolean conversion, null, undefined and ??.

Key points

- **Falsy:** false, 0, -0, 0n, "", null, undefined and NaN. Empty arrays and objects are truthy.

Practical example

```
console.log(Boolean([])); // true
console.log(Boolean(0));  // false
console.log(null ?? 'default');
```

Interview checkpoint: explain this topic in your own words and identify one real automation use case.

Page 9: Strings

Creation, immutability, indexing, template literals and common methods.

Key points

- **Strings are immutable. slice, includes, trim, replace and split are especially useful.**

Practical example

```
const text = ' Playwright ';  
console.log(text.trim().toUpperCase());  
console.log(`Hello ${'QA'}`);
```

Interview checkpoint: explain this topic in your own words and identify one real automation use case.

Page 10: Arrays

Creation, indexing, push/pop, shift/unshift, slice/splice and iteration.

Key points

- **Know mutation vs non-mutation. slice does not mutate; splice does.**

Practical example

```
const nums=[1,2,3];  
nums.push(4);  
console.log(nums.slice(1,3));
```

Interview checkpoint: explain this topic in your own words and identify one real automation use case.

Page 11: map, filter & reduce

Transformation, selection and accumulation with practical examples.

Key points

- **map transforms, filter selects, reduce accumulates. These three are core interview topics.**

Practical example

```
const nums=[1,2,3,4];
console.log(nums.map(n=>n*2));
console.log(nums.filter(n=>n%2===0));
console.log(nums.reduce((s,n)=>s+n,0));
```

Interview checkpoint: explain this topic in your own words and identify one real automation use case.

Page 12: find, some, every & includes

Searching and validation patterns.

Key points

- **find returns an element; some/every return booleans; includes checks a value.**

Practical example

```
const users=[{id:1,admin:false},{id:2,admin:true}];
console.log(users.find(u=>u.id===2));
console.log(users.some(u=>u.admin));
console.log(users.every(u=>'id' in u));
```

Interview checkpoint: explain this topic in your own words and identify one real automation use case.

Page 13: Objects

Properties, methods, dot/bracket notation and Object utility methods.

Key points

- **Objects model structured data. Object.keys/values/entries are common utility methods.**

Practical example

```
const user={name:'Asha',age:30};  
console.log(user.name);  
console.log(Object.keys(user));
```

Interview checkpoint: explain this topic in your own words and identify one real automation use case.

Page 14: Destructuring

Object and array destructuring, aliases, defaults and nested values.

Key points

- **Destructuring improves readability and is heavily used with API responses and function parameters.**

Practical example

```
const user={name:'Asha',age:30};  
const {name,age:userAge}=user;  
console.log(name,userAge);
```

Interview checkpoint: explain this topic in your own words and identify one real automation use case.

Page 15: Functions

Declarations, expressions, parameters, return values, callbacks and first-class functions.

Key points

- **Functions are first-class values. Understand parameters, return values and callbacks.**

Practical example

```
function add(a,b){return a+b;}  
const result=add(2,3);  
console.log(result);
```

Interview checkpoint: explain this topic in your own words and identify one real automation use case.

Page 16: Arrow Functions

Syntax, implicit return, callbacks and the difference in this behavior.

Key points

- **Arrow functions have lexical this and concise syntax. They are excellent for callbacks.**

Practical example

```
const add=(a,b)=>a+b;  
const square=n=>n*n;
```

Interview checkpoint: explain this topic in your own words and identify one real automation use case.

Page 17: Scope

Global, function, block and lexical scope.

Key points

- **Scope controls visibility. let/const are block-scoped; var is function-scoped.**

Practical example

```
function outer(){  
  const message='hello';  
  return ()=>message;  
}  
const fn=outer();  
console.log(fn());
```

Interview checkpoint: explain this topic in your own words and identify one real automation use case.

Page 18: Hoisting & TDZ

var hoisting, function declarations, let/const TDZ and ReferenceError.

Key points

- **Hoisting does not mean all declarations are safely usable before their line. let/const have a TDZ.**

Practical example

```
console.log(a); // undefined  
var a=10;
```

Interview checkpoint: explain this topic in your own words and identify one real automation use case.

Page 19: Closures

Lexical environment, retained state, factories, counters and practical uses.

Key points

- **A closure retains access to its lexical environment. It is a major JavaScript interview topic.**

Practical example

```
function counter(){let n=0;return ()=>{++n;}}
const next=counter();
console.log(next(),next());
```

Interview checkpoint: explain this topic in your own words and identify one real automation use case.

Page 20: this

Method calls, regular functions, arrow functions and context.

Key points

- **this depends on call-site for regular functions. Arrow functions inherit this.**

Practical example

```
const user={name:'Sam',show(){return this.name;}};  
console.log(user.show());
```

Interview checkpoint: explain this topic in your own words and identify one real automation use case.

Page 21: call, apply & bind

Controlling function context and passing arguments.

Key points

- **call/apply invoke immediately; bind returns a new bound function.**

Practical example

```
function greet(msg){return `${msg}, ${this.name}`;}  
const user={name:'Sam'};  
console.log(greet.call(user, 'Hello'));
```

Interview checkpoint: explain this topic in your own words and identify one real automation use case.

Page 22: Spread & Rest

Arrays, objects, function arguments, shallow-copy behavior.

Key points

- **Spread expands; rest collects. Both use ... but serve opposite purposes.**

Practical example

```
const copy=[...arr];  
const merged={...a,...b};  
function sum(...nums){return nums.reduce((s,n)=>s+n,0);}
```

Interview checkpoint: explain this topic in your own words and identify one real automation use case.

Page 23: Optional Chaining

Safe nested access and combining ?. with ??.

Key points

- **Optional chaining prevents errors when intermediate values are nullish.**

Practical example

```
const city=user?.address?.city ?? 'Unknown';
```

Interview checkpoint: explain this topic in your own words and identify one real automation use case.

Page 24: Conditional Statements

if/else, switch and ternary operator.

Key points

- **Choose if/else, switch or ternary based on readability and complexity.**

Practical example

```
const status=active ? 'Active' : 'Inactive';
```

Interview checkpoint: explain this topic in your own words and identify one real automation use case.

Page 25: Loops

for, while, do/while, for...of, for...in, break and continue.

Key points

- Use for...of for iterable values. for...in is primarily for object keys.

Practical example

```
for(const item of items){console.log(item);}
for(const key in obj){console.log(key);}
```

Interview checkpoint: explain this topic in your own words and identify one real automation use case.

Page 26: Date & Math

Date objects, timestamps and useful Math methods.

Key points

- **Be explicit about time zones in tests. Math provides common numeric helpers.**

Practical example

```
const now=new Date();  
console.log(Math.max(10,20,30));  
console.log(Math.floor(4.9));
```

Interview checkpoint: explain this topic in your own words and identify one real automation use case.

Page 27: Exception Handling

try, catch, finally, throw and error propagation.

Key points

- **Catch errors only when you can handle or enrich them. Finally is useful for cleanup.**

Practical example

```
try{riskyOperation();}catch(error){console.error(error.message);}finally{cleanup();}
```

Interview checkpoint: explain this topic in your own words and identify one real automation use case.

Page 28: Classes

Constructors, methods, instances and static members.

Key points

- **Classes package state and behavior. Constructors initialize instance state.**

Practical example

```
class User{constructor(name){this.name=name;}greet(){return this.name;}}
```

Interview checkpoint: explain this topic in your own words and identify one real automation use case.

Page 29: Inheritance

extends, super, overriding and practical guidance.

Key points

- **extends and super implement inheritance. Method overriding enables specialized behavior.**

Practical example

```
class Admin extends User{constructor(name){super(name);this.role='admin';}}
```

Interview checkpoint: explain this topic in your own words and identify one real automation use case.

Page 30: Encapsulation

Private # fields, controlled access and object invariants.

Key points

- # fields provide language-level private class state.

Practical example

```
class Account{#balance=0;deposit(n){this.#balance+=n;}getBalance(){return this.#balance;}}
```

Interview checkpoint: explain this topic in your own words and identify one real automation use case.

Page 31: Polymorphism

Common interfaces, overriding and reusable functions.

Key points

- **Polymorphism lets code call a shared operation while implementations provide different behavior.**

Practical example

```
function speak(animal){return animal.sound();}
```

Interview checkpoint: explain this topic in your own words and identify one real automation use case.

Page 32: Callbacks

Passing functions, asynchronous callbacks and callback complexity.

Key points

- **Callbacks are functions passed into other functions. Promises generally make async flows easier to compose.**

Practical example

```
function process(data, callback){callback(data);}
process('test', value=>console.log(value));
```

Interview checkpoint: explain this topic in your own words and identify one real automation use case.

Page 33: Promises

States, then/catch/finally and Promise.all/allSettled/race.

Key points

- **Promises represent eventual completion. Know all, allSettled, race and reject behavior.**

Practical example

```
Promise.all([getUser(),getOrders()])  
  .then(([user,orders])=>console.log(user,orders));
```

Interview checkpoint: explain this topic in your own words and identify one real automation use case.

Page 34: async/await

Async functions, await, error handling and concurrency.

Key points

- **async functions return promises. await makes asynchronous code easier to read.**

Practical example

```
async function load(){  
  const response=await fetch(url);  
  return response.json();  
}
```

Interview checkpoint: explain this topic in your own words and identify one real automation use case.

Page 35: Event Loop

Call stack, runtime APIs, microtasks, tasks and execution order.

Key points

- **Understand stack, microtasks and tasks. Promise handlers are microtasks.**

Practical example

```
console.log('A');  
Promise.resolve().then(()=>console.log('microtask'));  
setTimeout(()=>console.log('task'), 0);  
console.log('B');
```

Interview checkpoint: explain this topic in your own words and identify one real automation use case.

Page 36: Modules

ES modules, import/export, default vs named exports.

Key points

- **Modules provide explicit dependencies and reusable code boundaries.**

Practical example

```
export const add=(a,b)=>a+b;  
import {add} from './math.js';
```

Interview checkpoint: explain this topic in your own words and identify one real automation use case.

Page 37: JSON

JSON.parse, JSON.stringify, API payloads and limitations.

Key points

- **JSON is common for APIs. JSON is text; JavaScript objects are runtime values.**

Practical example

```
const json=JSON.stringify({id:1});  
const obj=JSON.parse(json);
```

Interview checkpoint: explain this topic in your own words and identify one real automation use case.

Page 38: Fetch & REST APIs

HTTP requests, status validation, JSON response handling and testing.

Key points

- **Always inspect HTTP status where appropriate. Parse response data only after validating the response.**

Practical example

```
const response=await fetch(url);
if(!response.ok)throw new Error(`HTTP ${response.status}`);
const data=await response.json();
```

Interview checkpoint: explain this topic in your own words and identify one real automation use case.

Page 39: Node.js Fundamentals

process, environment variables, fs, path and CLI execution.

Key points

- **Node provides runtime APIs and process/environment information used by test frameworks.**

Practical example

```
console.log(process.env.BASE_URL);
```

Interview checkpoint: explain this topic in your own words and identify one real automation use case.

Page 40: npm & package management

Dependencies, devDependencies, scripts, semantic versions and lockfiles.

Key points

- **Lockfiles improve repeatable installs. Keep test tooling in devDependencies when appropriate.**

Practical example

```
npm install @playwright/test
npm test
```

Interview checkpoint: explain this topic in your own words and identify one real automation use case.

Page 41: JavaScript in Playwright

Async operations, locators, assertions, fixtures and reliable automation.

Key points

- **Playwright APIs are asynchronous. Await navigation, actions and web-first assertions.**

Practical example

```
test('login', async({page})=>{  
  await page.goto(url);  
  await page.getByLabel('Username').fill('user');  
  await expect(page).toHaveURL(/dashboard/);  
});
```

Interview checkpoint: explain this topic in your own words and identify one real automation use case.

Page 42: Playwright Page Objects

Encapsulation, reusable actions, locators and maintainable tests.

Key points

- **Page Objects centralize locators and business actions, reducing duplication.**

Practical example

```
class LoginPage{
  constructor(page){this.page=page;}
  async login(u,p){
    await this.page.getByLabel('Username').fill(u);
    await this.page.getByLabel('Password').fill(p);
  }
}
```

Interview checkpoint: explain this topic in your own words and identify one real automation use case.

Page 43: Playwright API Testing

APIRequestContext, request bodies, response validation and negative tests.

Key points

- **API tests validate status, headers, payloads, business rules and negative scenarios.**

Practical example

```
const response=await request.post('/users',{data:{name:'John'}});  
expect(response.status()).toBe(201);
```

Interview checkpoint: explain this topic in your own words and identify one real automation use case.

Page 44: Async pitfalls in automation

Missing await, race conditions, sequential vs concurrent work.

Key points

- **Missing await is a common cause of race conditions and misleading test results.**

Practical example

```
await page.goto(url);  
await expect(page).toHaveURL(/login/);
```

Interview checkpoint: explain this topic in your own words and identify one real automation use case.

Page 45: Error handling in automation

Diagnostics, screenshots, traces, rethrowing and avoiding false positives.

Key points

- **Preserve diagnostics and rethrow when a test should fail. Avoid swallowing exceptions.**

Practical example

```
try{await action();}catch(e){console.error(e);throw e;}
```

Interview checkpoint: explain this topic in your own words and identify one real automation use case.

Page 46: Clean JavaScript

Naming, small functions, immutability, early returns and avoiding global state.

Key points

- **Readable code is maintainable code: small functions, good names and minimal shared mutable state.**

Practical example

```
const getActiveUsers=users=>users.filter(u=>u.active);
```

Interview checkpoint: explain this topic in your own words and identify one real automation use case.

Page 47: Debugging

console tools, debugger, VS Code, Playwright Inspector and trace debugging.

Key points

- **Use breakpoints, debugger, Playwright Inspector and traces for systematic diagnosis.**

Practical example

```
debugger;  
console.log({user, response});
```

Interview checkpoint: explain this topic in your own words and identify one real automation use case.

Page 48: Interview Questions

High-frequency questions and how to answer with definition, example and pitfall.

Key points

- **For interviews: definition → example → practical use → pitfall is a strong answer structure.**

Practical example

```
// Explain: closure, hoisting, event loop, this, promises, async/await
```

Interview checkpoint: explain this topic in your own words and identify one real automation use case.

Page 49: Coding Exercises

Reverse string, duplicates, frequency, second-largest, flattening, grouping and more.

Key points

- **Solve problems without copying solutions. Then explain time/space complexity where relevant.**

Practical example

```
const unique=[...new Set([1,2,2,3])];  
const counts={};  
for(const ch of 'banana')counts[ch]=(counts[ch]||0)+1;
```

Interview checkpoint: explain this topic in your own words and identify one real automation use case.

Page 50: Final Revision & Cheat Sheet

Core syntax, interview checklist and Playwright-focused JavaScript revision.

Key points

- **Revise the language core first, then connect each concept to Playwright and API automation.**

Practical example

```
// Revision: variables -> types -> arrays/objects -> functions -> scope
// -> closures/this -> promises -> async/await -> event loop
// -> modules/Node.js -> Playwright automation
```

Interview checkpoint: explain this topic in your own words and identify one real automation use case.